

GEOPOLITICAL THREAT PREDICTOR

A Weighted Three-Parameter Risk Index in Python

VITyarthi Project Report

Author: Rajanya Sharma

Programme: B.Tech, Aerospace Engineering

Institution: VIT Bhopal University

Application: Python terminal program

Repository: github.com/rajanya26bas10048-debug/Geopolitical-Analyser

1. Introduction

- Geopolitical Threat Predictor is a small Python program that turns three ratings into one Threat Index between 10 and 100.
- The user rates inflation, border tension and resource scarcity from 1 to 10. The program combines them with fixed weights and reports a risk category.
- It is an educational model of weighted scoring. It is not a forecasting tool, and it uses no dataset, machine learning or statistics.

Objectives

- Represent three conditions as numbers
- Apply a weighted linear formula to them
- Map the score to a category using if/elif/else

2. Problem Statement

- A geopolitical situation depends on many factors, and they do not all matter equally. A single number is hard to justify unless each factor has a stated importance.
- This project narrows the problem to three indicators and gives each an explicit weight, so the calculation can be read and checked by anyone.
- The weights, thresholds and category names are assumptions chosen for the model. They are not official measurements.

Scope: one run per execution, three typed inputs, one weighted formula, one printed report.

3. Functional Requirements

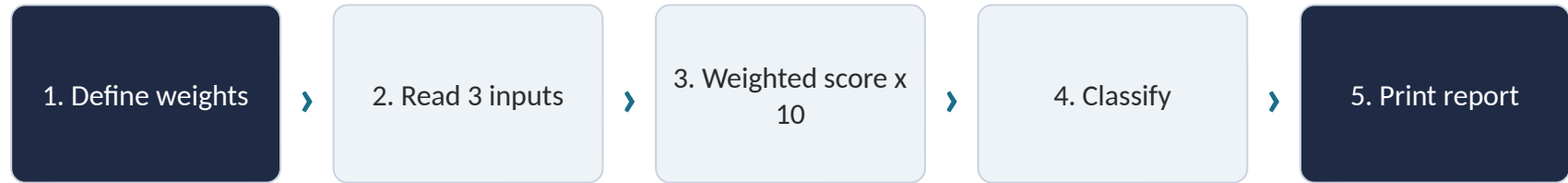
ID	Requirement
FR-01	Accept inflation, border tension and resource scarcity as decimal numbers.
FR-02	Store the weights 0.20, 0.30 and 0.50 as named constants.
FR-03	Compute the weighted sum of the three inputs.
FR-04	Multiply the sum by 10 to get the Threat Index on a 10 to 100 scale.
FR-05	Classify the index: 75 or above is Critical Risk, 45 or above is Moderate Risk, otherwise Stable.
FR-06	Print the index to two decimals along with the category message.

4. Non-functional Requirements

Requirement	How the program meets it
Simplicity	One short script. Each step is numbered in the comments.
Transparency	Weights are named constants at the top, so assumptions are visible.
Repeatability	The same three inputs always give the same output.
Portability	Runs on any Python 3 setup. No libraries are imported.
Usability	Prompts state the 1 to 10 scale. The report has a fixed layout.

5. System Architecture

The program is one Python file that runs top to bottom in five stages.

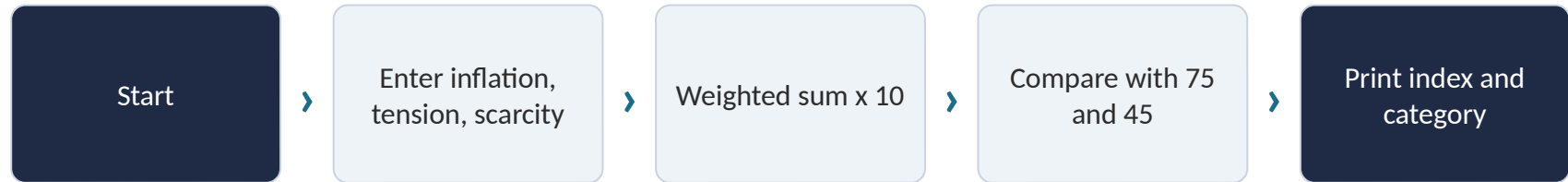


Files: a single script, with no modules, database or external storage.

Note: the project is a single file. A future version could be split into modules (see Future Enhancements).

6. Design Diagrams

Program workflow



Classification decision

Condition (checked in this order)	Category shown
index $\geq$ 75.0	CRITICAL RISK
index $\geq$ 45.0 (and below 75)	MODERATE RISK
otherwise (below 45)	STABLE

7. UML / Component Design

- The implementation is procedural. It has no classes or functions, so a class diagram would not describe it accurately.
- Its components are five blocks of code in sequence:

Component	Role
Weight constants	WEIGHT_INFLATION, WEIGHT_TENSION, WEIGHT_SCARCITY
Input block	Three input() calls converted with float()
Scoring block	raw_weighted_score, then final_threat_index
Classification block	if / elif / else sets risk_category
Report block	Prints the index and the conclusion

ER diagram: not applicable, because the program stores no data.

8. Design Decisions & Rationale

Decision	Rationale
Three indicators	Keeps the model small enough to explain and check by hand.
Weights 0.20 / 0.30 / 0.50	Gives scarcity the largest say and inflation the smallest. The weights add up to 1.0.
Linear weighted sum	Easy to calculate and to interpret for each input.
Multiply by 10	Turns the 1 to 10 scale into a 10 to 100 index that reads like a percentage.
Thresholds at 45 and 75	Splits the range into three bands: Stable, Moderate Risk and Critical Risk.
Named constants	Weights can be changed in one place without touching the formula.

9. Implementation Details

Item	Value
WEIGHT_INFLATION	0.20
WEIGHT_TENSION	0.30
WEIGHT_SCARCITY	0.50
Input scale	1.0 (low risk) to 10.0 (extreme risk)
Critical Risk threshold	75.0 and above
Moderate Risk threshold	45.0 and above

Formulas

Weighted Score =
 $(I \times 0.20) + (T \times 0.30) + (S \times 0.50)$

Threat Index = Weighted Score x 10

I = inflation, T = border tension, S = resource scarcity

Because the inputs run from 1 to 10 and the weights add up to 1, the index always falls between 10 and 100.

10. Screenshots / Results

Worked example: inflation = 6, border tension = 8, resource scarcity = 7

Weighted score = $(6 \times 0.20) + (8 \times 0.30) + (7 \times 0.50) = 1.2 + 2.4 + 3.5 = 7.1$

Threat Index = $7.1 \times 10 = 71.00$, which is 45 or above and below 75, so the category is Moderate Risk.

```
=====
                        COMPUTATIONAL RISK REPORT
=====
Calculated Threat Index : 71.00 / 100.00
Algorithmic Conclusion  : MODERATE RISK: Elevated instability,
                        diplomatic intervention required
=====
```

11. Testing Approach

The project has no automated test suite. It was checked by running it with chosen inputs and comparing the output with a hand calculation.

Inputs (I, T, S)	Hand calculation	Expected result
2, 2, 2	$(0.4 + 0.6 + 1.0) \times 10$	20.00, Stable
6, 8, 7	$(1.2 + 2.4 + 3.5) \times 10$	71.00, Moderate Risk
9, 9, 9	$(1.8 + 2.7 + 4.5) \times 10$	90.00, Critical Risk
10, 10, 10	10×10	100.00, Critical Risk

Limitation: the program does not check its input. Text, or numbers outside 1 to 10, are not handled, and this was not tested.

12. Challenges Faced

- Reducing a complicated real-world problem to three numbers, without pretending the result is more than a simplified model.
- Choosing weights. They are judgement calls, so they are documented as assumptions and kept as named constants.
- Turning a continuous score into categories. The 45 and 75 cut-offs are chosen, not derived from data.
- Dropping an earlier idea of matching against past cases, which made the code harder to explain, in favour of a direct formula.
- Limitation: the result depends entirely on the user's own ratings, so different users can get different indexes for the same situation.

13. Learnings & Key Takeaways

- Turning a qualitative idea into numbers that a program can process.
- Using a weighted linear combination to merge several factors into one score.
- Reading user input with `input()` and converting it with `float()`.
- Using `if / elif / else` in the right order to sort a score into categories.
- Keeping assumptions in named constants so they are easy to see and change.
- Understanding that a clear formula is not the same as a validated prediction.

14. Future Enhancements

- Validate input: reject text and values outside 1 to 10, and ask again.
- Add a loop so several analyses can be run without restarting.
- Move the scoring and classification into functions, then split the code into modules.
- Write unit tests for the formula and the category boundaries.
- Add more indicators, such as political stability or trade dependence.
- Justify the weights and thresholds using real data before drawing any real-world conclusions.

15. References

- VITyarthi: Build Your Own Project, General Project Instructions & Submission Guidelines.
- Project repository: github.com/rajanya26bas10048-debug/Geopolitical-Analyser
- Python documentation: `input()`, `float()`, f-strings and `if / elif / else` (docs.python.org).